

Ros2 Day2 hw3 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 코드 구조
3. 파일 구조
4. 실행결과

1. 프로젝트 개요

이 프로젝트는 Qt로 만든 창 두 개로 간단한 채팅을 주고받는 프로그램이다. 한쪽 창에서 글자를 입력해서 보내면 다른 쪽 창에 그 글자가 표시된다.

hw2에서는 실행 파일이 하나였지만 이번에는 보내는 쪽과 받는 쪽을 따로 만들어 실행 파일을 두 개로 나누었다. 두 프로그램은 같은 이름의 토픽으로 연결되어 있고, 주고받는 데이터는 문자열이라 기본으로 제공되는 메시지 타입을 그대로 사용했다.

보내는 쪽 창에는 글자를 입력하는 칸과 버튼을 두었다. 버튼을 누르면 칸에 적힌 글자가 발행되고 입력 칸은 비워진다. 받는 쪽 창에는 라벨 하나만 두었고, 메시지가 들어올 때마다 그 내용으로 바뀐다.

패키지는 `ros2_create_qt_pkg` 스크립트로 생성했다. ROS2와 Qt를 같이 쓰는 패키지는 빌드

설정을 처음부터 잡기가 번거로운데, 이 스크립트가 폴더 구조와 CMakeLists의 기본 틀을 만들어준다. 여기에 hw2에서 만든 것과 같은 방식으로 필요한 부분을 채워 넣었다. ROS 통신을 담당하는 부분과 화면을 담당하는 부분은 hw2와 마찬가지로 나누어 작성했다. 받는 쪽은 메시지가 들어오면 통신 쪽이 화면 쪽으로 신호를 보내 라벨을 갱신하도록 했다.

2. 코드 구조

```
TalkerNode::TalkerNode()
{
    ...
    node = rclcpp::Node::make_shared("talker_node");
    talker_ = node->create_publisher<std_msgs::msg::String>("chat_topic", 10);
    this->start();
}
```

보내는 쪽 노드를 만들고 발행자를 등록한 뒤 스레드를 시작하는 부분

```
void TalkerNode::publish_msg(std::string text)
{
    auto msg = std_msgs::msg::String();
    msg.data = text;
    talker_->publish(msg);
    RCLCPP_INFO(node->get_logger(), "sent: %s", text.c_str());
}
```

받은 글자를 메시지에 담아 발행하고 로그를 남기는 부분

```
connect(ui->btn_send, &QPushButton::clicked, this, [this]() {
    qnode->publish_msg(ui->line_input->text().toStdString());
    ui->line_input->clear();
});
```

버튼이 눌리면 입력 칸의 글자를 가져와 발행하고 칸을 비우는 부분. Qt에서 쓰는 문자열과 ROS 메시지에 들어가는 문자열이 서로 다른 타입이라 변환해서 넘겨준다.

```
ListenerNode::ListenerNode()
```

받는 쪽 노드를 만들고 같은 토픽을 구독하면서 콜백을 연결하는 부분

```

void ListenerNode::msg_callback(const std_msgs::msg::String::SharedPtr msg)
{
    RCLCPP_INFO(node->get_logger(), "recv: %s", msg->data.c_str());
    Q_EMIT msgReceived(QString::fromStdString(msg->data));
}

```

메시지가 들어오면 로그를 남기고 화면 쪽으로 신호를 보내는 부분. 여기서는 반대로 ROS 쪽 문자열을 Qt에서 쓰는 형태로 바꿔서 넘긴다. 통신 쪽 스레드에서 라벨을 직접 건드릴 수 없기 때문에 신호로 넘기는 방식을 썼다.

```

void ListenerNode::run()

```

스레드에서 노드를 계속 돌려주는 부분. 이게 있어야 콜백이 실행된다.

```

connect(qnode, &ListenerNode::msgReceived, this, &ListenerWindow::updateRecvLabel);

```

통신 쪽에서 온 신호를 라벨 갱신 함수에 연결하는 부분

```

void ListenerWindow::updateRecvLabel(QString text)
{
    ui->label_recv->setText(text);
}

```

받은 글자를 라벨에 표시하는 부분. hw2와 달리 대기하는 코드가 없어서 화면이 멈추지 않으므로, 이벤트를 강제로 처리해줄 필요는 없었다.

```

add_executable(talker
    src/talker_main.cpp
    src/talker_window.cpp
    src/talker_qnode.cpp
    ${TALKER_MOC}
    ${TALKER_UI}
)
...
add_executable(listener
    ...
)

```

실행 파일을 두 개로 나누어 만드는 부분. 각각에 필요한 소스와 ui 파일만 넣어주었다.

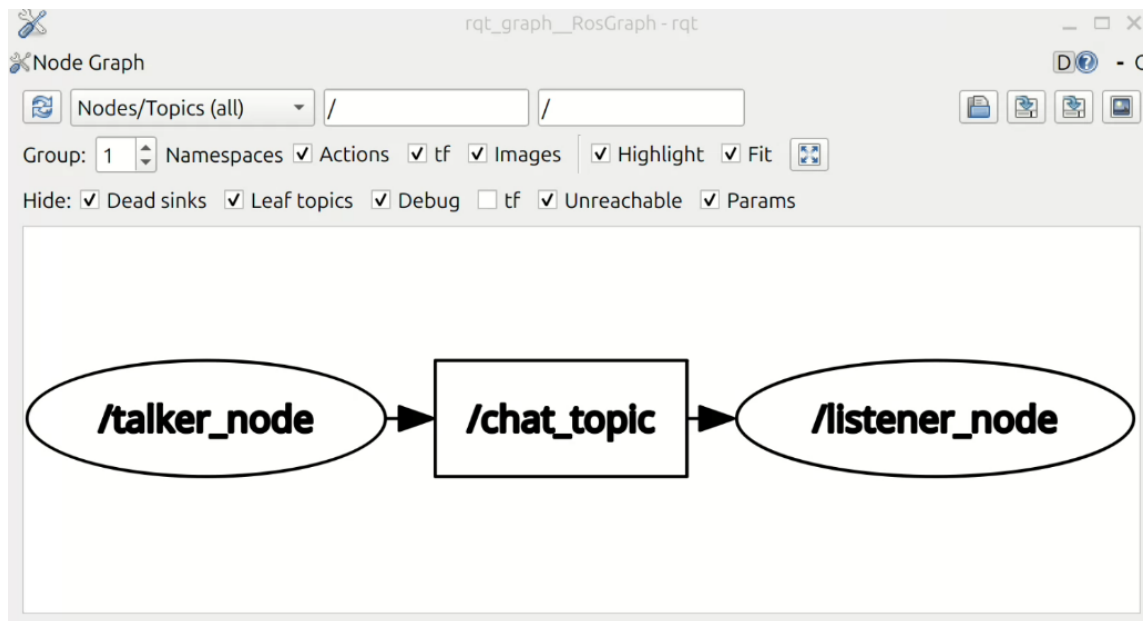
3.파일 구조

hw3

```
|— build
|— install
|— log
└— src
    └— juwon_chat
        |— include
        |   └— juwon_chat
        |       |— listener_qnode.hpp
        |       |— listener_window.hpp
        |       |— talker_qnode.hpp
        |       └— talker_window.hpp
        |— src
        |   |— listener_main.cpp
        |   |— listener_qnode.cpp
        |   |— listener_window.cpp
        |   |— talker_main.cpp
        |   |— talker_qnode.cpp
        |   └— talker_window.cpp
        |— ui
        |   |— listener.ui
        |   └— talker.ui
        |— resources
        |— CMakeLists.txt
        └— package.xml
```

기본 구조는 `ros2_create_qt_pkg`로 생성된 것을 그대로 두고, 그 안에서 보내는 쪽과 받는 쪽 파일을 이름으로 구분해 채워 넣었다. 각각 통신을 담당하는 파일과 화면을 담당하는 파일, 그리고 시작점이 되는 파일로 나뉘어 있고, 창의 배치는 `ui` 폴더의 파일에 들어 있다.

4. 실행결과



특 노드가 리스너 노드로 챗 토픽을 통해 대화를 전해주는 모습을 볼 수 있다.

첨부 영상 참고